CSC 340
Programming Methodology
Final Review

Joseph Hui
jhui@sfsu.edu

CSC 340 Final Format

- Multiple Choice and True/False (25 questions – 1 point each)
  - ~10 from Midterm I
  - ~10 from Midterm II
  - ~5 from Weeks 13-16

- Essay Questions (5 questions – 5 points each)
  - 2 from Midterm I
  - 2 from Midterm II
  - 1 from Weeks 13-16

- Goal is to make it close to real world example (Interview Questions)

CSC 340 Final Content

- From Midterm I:
  - Problem Solving
  - C++: Syntax, memory management, STL, C++ versus Java
  - Data Structures: C++ implementation, Complexity of Algorithms, STL

- From Midterm II:
  - Design Patterns
  - Cache Manager
  - Bubble Sort
  - BST

- From Weeks 13-16
  - Graphs
  - Merge Sort

Data Structure Overview

- Provides approach to store, organize and retrieve data efficiently
- Basic types: array, linked list
- Advanced types: tree, hash table
- Certain data structures lend themselves to certain applications
  - UI that will display 25 rows of data?
  - Cache search by key (no FIFO)?
- Data structures + algorithms = programs

When to use a particular Data Structure

- Go over scenarios and decide which Data Structure to use

- Scenarios will be from Job Interview Questions
  - Almost all interviews for software development have Data Structure questions
  - My personal favorites, as well as interview questions on web

- The one skill I want you to develop in this class, is the ability to analyze a problem and determine which Data Structure(s) to use to solve it.
  - Good for Job Interviews
  - But, more importantly, it’s good to know for your career

Data Structures

- Question will supply a problem
  - Data Structure will need to be recommended
  - Algorithm will need to be recommended (i.e., is recursion suitable)

- Array

- Linked List

- Hash Table

- Trees

Array

- Contiguous area of memory containing multiple data of the same type

- Pros
  - O(1) access by index

- Cons
  - Static data size and expensive to expand

Linked List

- Linear data structure consisting of nodes connected by pointers

- Pros
  - Dynamic sizing with insert/update/delete is straightforward

- Cons
  - Finding a particular key is O(n)

Hash Table (Dictionary)

- Major components:
  - Array of buckets
  - Hashcode to determine index into Array
  - Collision resolution algorithm

- Pros
  - O(1) access via key

- Cons
  - No ordered retrieval

Tree

- Hierarchical structure of nodes
  - One parent
  - Zero or more children

- BST Pros
  - Efficient searching
  - Fast insertion and deletion O(log n)

- BST Cons
  - Memory overhead
  - Imbalanced trees
  - Complexity - recursion

Binary Search Tree

- Binary Tree Traversal
  - Preorder
  - Inorder – sorted order traversal
  - Postorder
  - Pseudocode for each of these traversals
- Complete/Full tree definitions
- Searching Binary Search Tree Algorithm
  - Pseudocode for searching a BST

Sample Data Structure Interview Questions

- What criteria do you use to determine which Data Structure to use?

- What Data Structure would use to:
  - Lookup a unique key as quickly as possible?
  - Keep a sorted order for data?
  - What Data Structures supply O(1) access?
  - What Data Structure(s) would you use for a Cache Manager?
    - What is the reason for each of the Data Structures?

Object-Oriented Concepts

- Object-oriented analysis and design (OOAD)
  - Process for solving problems
- Solution
  - Computer program consisting of system of interacting classes of objects
- Object
  - Has set of characteristics, behaviors related to solution

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Cohesion

- Each module should perform one well-defined task

- Benefits
  - Well named, self-documenting
  - Easy to reuse
  - Easier to maintain
  - More robust

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Coupling

- Measure of dependence among modules
- Dependence
  - Sharing data structures or calling each other's methods
- Modules should be loosely coupled
  - Highly coupled modules should be avoided

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

UML Diagrams

- UML has many different diagram types
  - Sequence, State, Timing, Use Case, etc.

- In industry, most often used is the Class Diagram
  - Useful during analysis and design
  - Finished diagram can also be used as documentation

- Become familiar with the Class Diagram!

Important Class Diagram artifacts

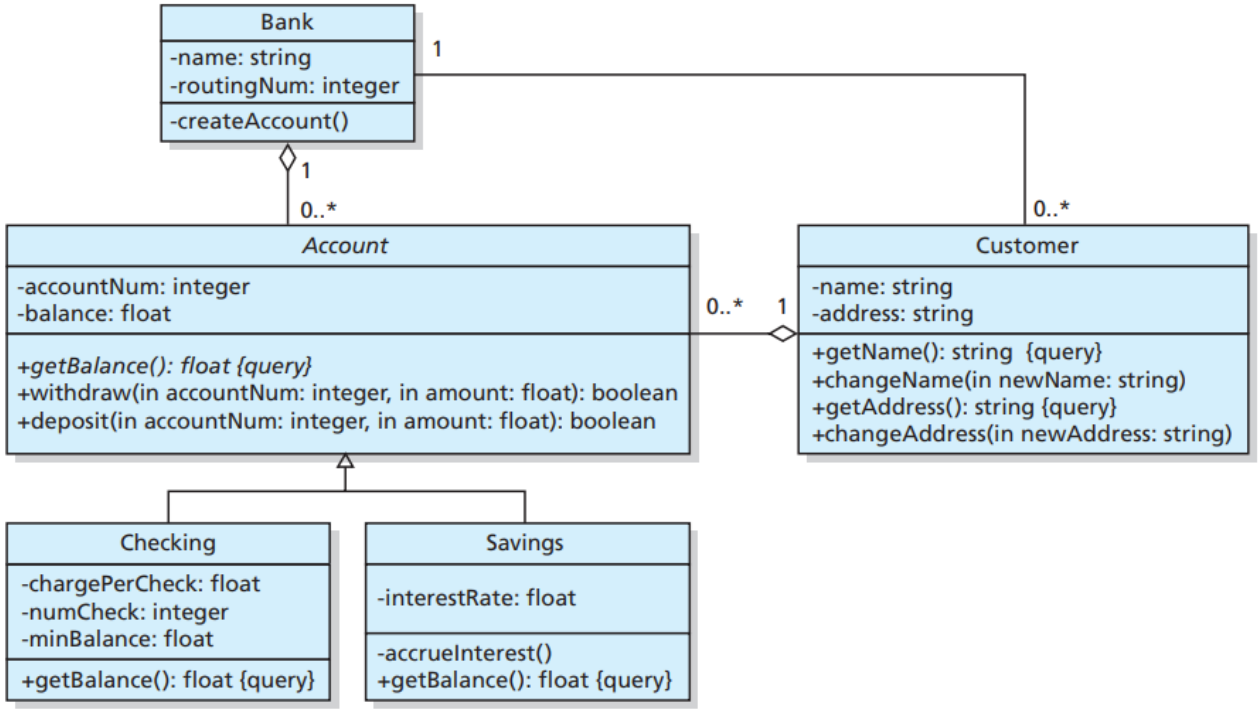
- Class block - Name, attributes, methods

- Association – solid line with no arrowheads
  - Multiplicities – numbers at the end of the lines to show cardinality

- Aggregation – open diamond
  - Stand alone, are not joined at the hip; Bank can exist without an Account

- Composition – filled-in diamond
  - Must appear together, Ownership; Room can’t exist without a House

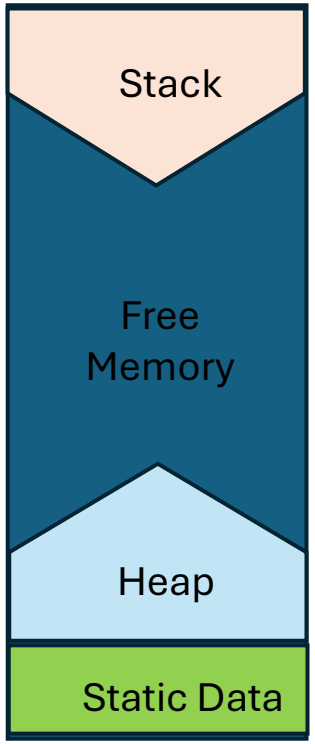
Class Diagram Example – Bank Accounts



Memory

- Memory distribution in a typical operating system process

- Stack – the stack is for local variables and for maintaining a record of function calls. The stack grows from the top of memory down towards the heap.
- Heap - The heap is for dynamically allocated data items. The heap grows from the top of the static data area up as data items are allocated.
- Static Data - This is a block of reserved space in RAM for all the global and static variables from your program. Allocated once and lasts for duration of a program.



Automatic, static, and dynamic data in Java and C++

- A running program typically has 3 kinds of data in it:
  - automatic
  - static
  - dynamic

- These kinds of data differ in:
  - what region of machine memory the data is stored in
  - when and how memory for the data is allocated (“created”)
  - when and how memory for the data is deallocated (“destroyed”)

- Both Java and C++ have basically these same 3 kinds of data, with some differences

- (Note: these are kinds of data, i.e. variables. Executable instructions are stored somewhat differently.)

Automatic data

- Automatic data is called "automatic" because it is automatically created and destroyed when your program runs
  - Examples of automatic data in both Java and C++: formal parameters to functions, local variables in functions (if the variables are not declared static)
- Memory for automatic data is allocated on the runtime stack
- Memory for automatic data is allocated when execution reaches the scope of the variable's declaration
- Memory for automatic data is deallocated when execution leaves that scope

Static data

- Static data is called "static" because it essentially exists during the entire execution of your program
  - Examples of static data in Java and C++: variables declared static (or, in C++, variables declared outside any function and class body)
- Memory for static data is allocated in the static data segment
- Memory for static data is allocated when your program starts, or when execution reaches the static data declaration for the first time
- Memory for static data is deallocated when your program **exits**

Dynamic data

- Dynamic data is called "dynamic" because it is allocated (and, in C/C++, destroyed) under explicit programmer control
  - Example of dynamic data in Java: *all* objects (instances of a class), including arrays. Dynamic data is created using the **`new`** operator
  - Example of dynamic data in C++: any variables created using the **`new`** operator. In C++, any type variable can be created using **`new`**
- Memory for dynamic data is allocated from the "heap"
- In Java, dynamic data exists from the time **`new`** is called until the object is reclaimed by the garbage collector
- In C++, dynamic data exists from the time it is allocated with the **`new`** operator until it is deallocated with the **`delete`** operator

Dynamic data bugs

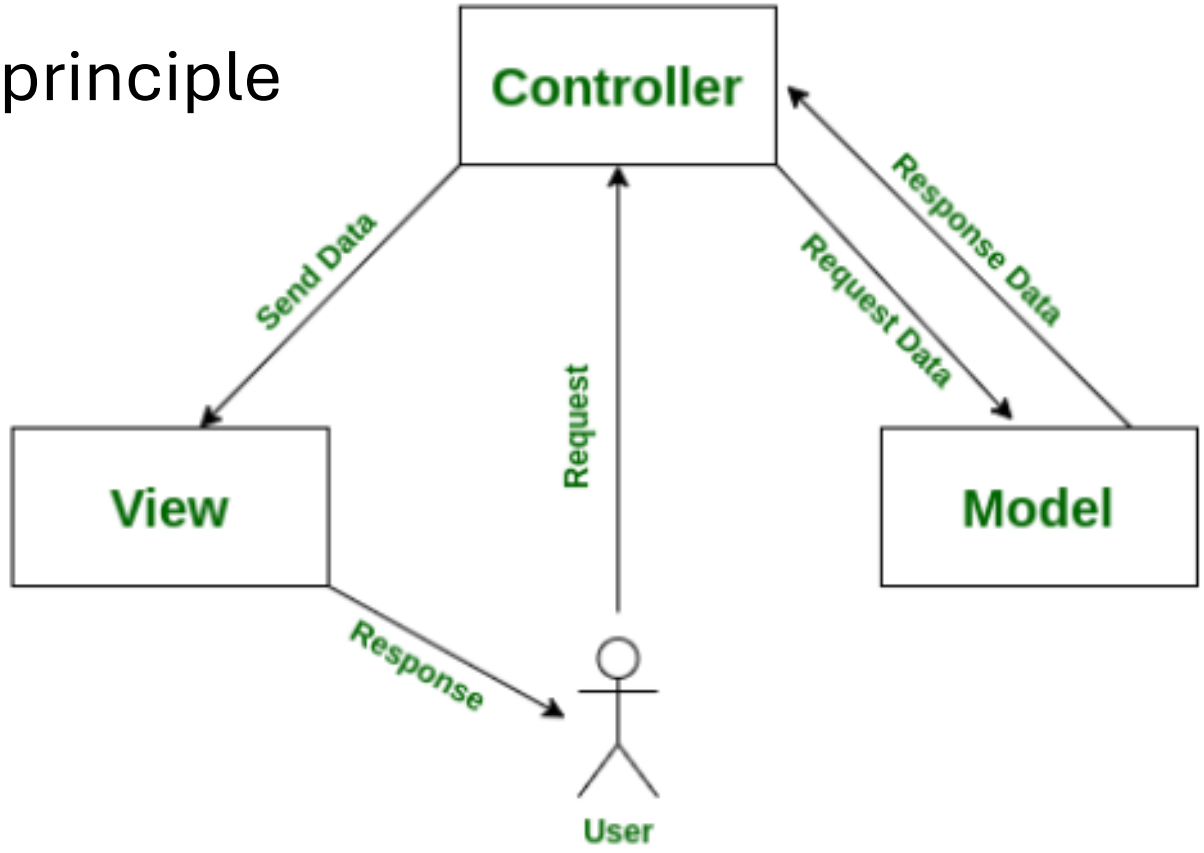
- Automatic and static data is created and destroyed automatically. As a programmer, you should know how and when this happens, but you do not need to take steps to make it happen
- Because the standard C++ runtime environment does not have a garbage collector, both allocation and deallocation of *dynamic* data must be done under programmer control
- This leads to common bugs that can be very hard to track down.
- We'll look at these types of bugs:
  - Inaccessible objects
  - Memory leaks
  - Dangling pointers

Data Structures and Design Patterns

- Data Structures are the foundation of Software Development
  - All engineers that develop software must know Data Structures
- Modern software cannot be developed without Data Structures
- Data Structures are the grammar of software
  - Grammar means the basics that all software are based on for development
- Products are based on Design Patterns
  - All software products are based on Design Patterns
  - All Design Patterns are based on Data Structures

Behavioral - MVC

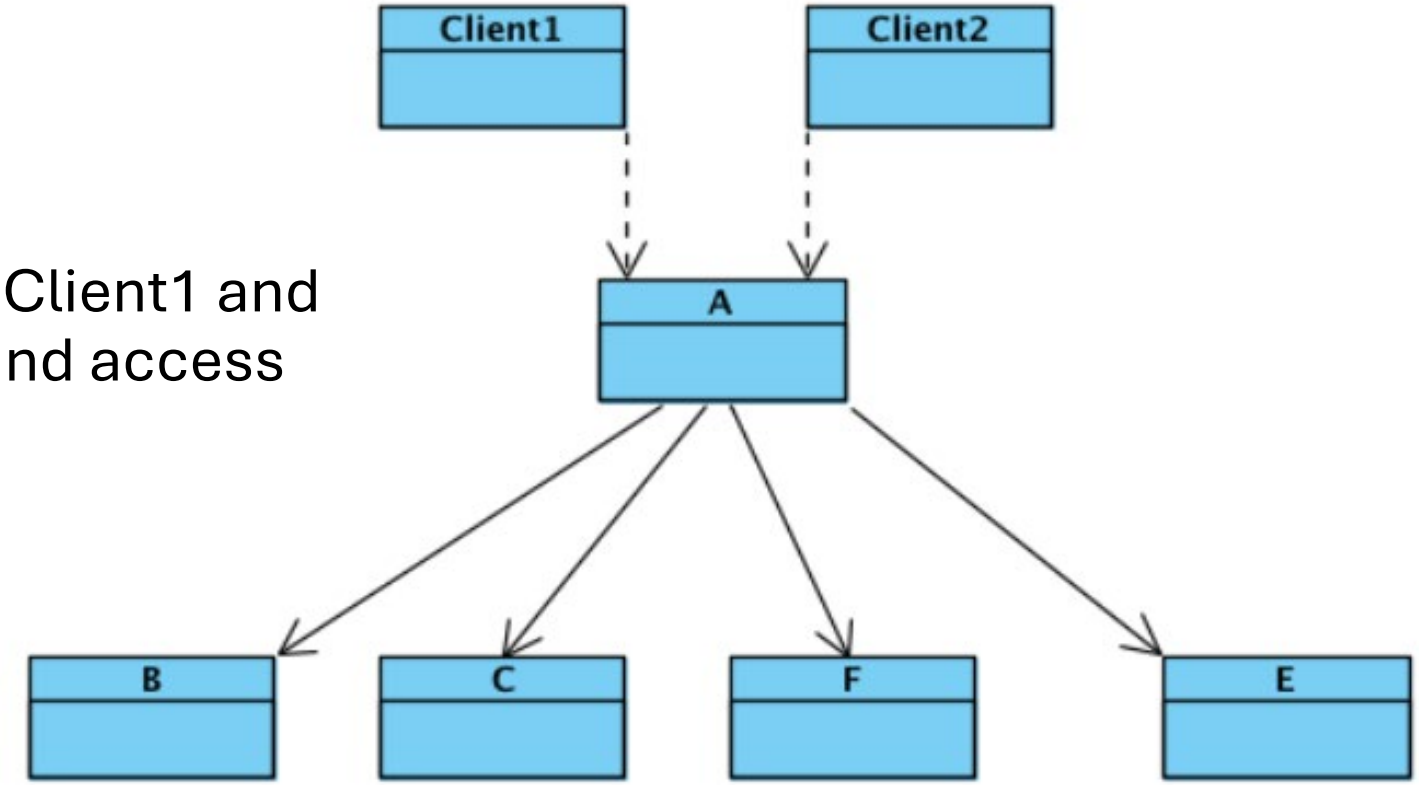
- Developed at PARC in late 70s
- Used by almost all modern UI's
- Separation of Concerns design principle
  - Distinct sections of functionality



Behavioral - Façade

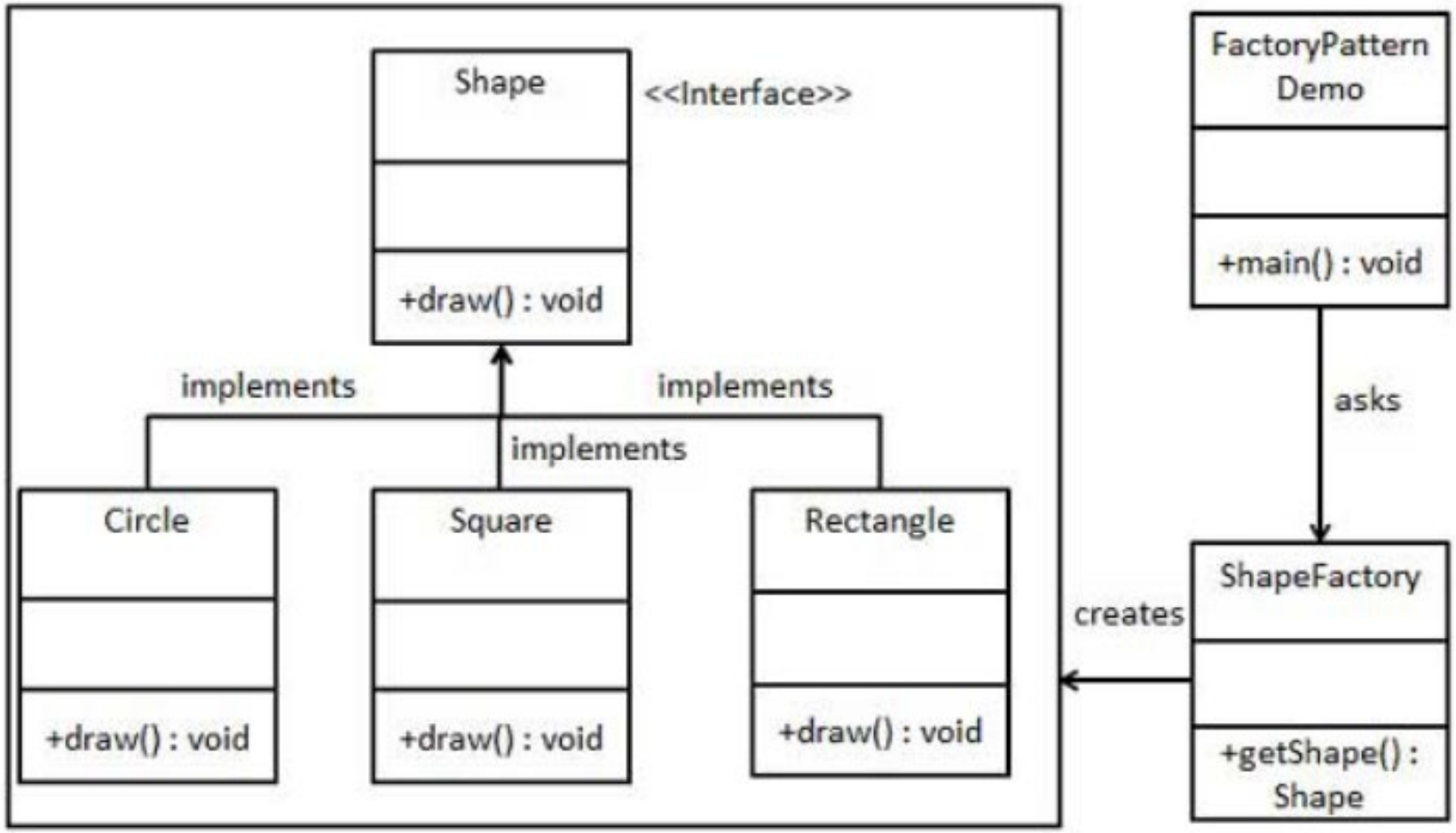
- JDBC is a good example

- A is the JDBC façade
- B, C, F, and E are different implementations
  - Allows different clients (i.e., Client1 and Client2) to be written once and access multiple databases

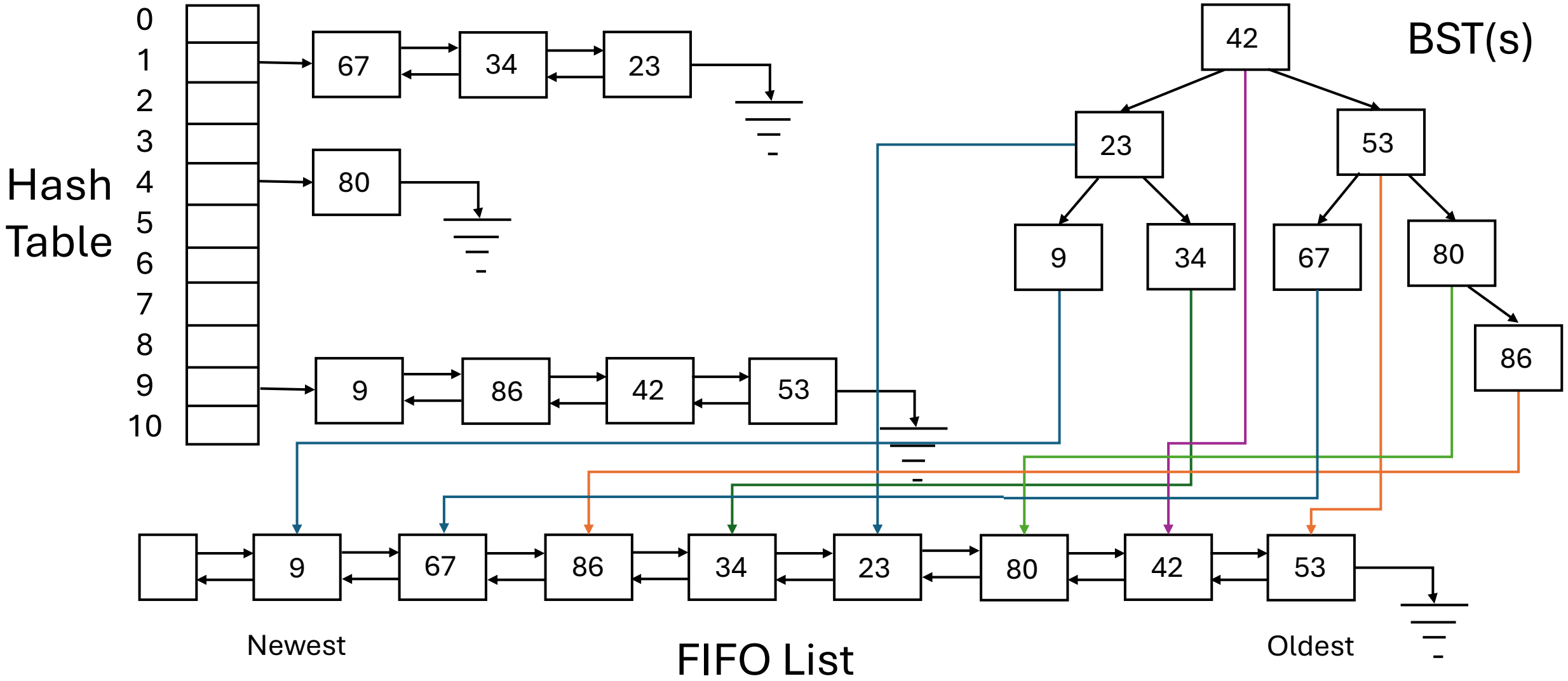


Creational - Factory

- Factory chooses which subclass to instantiate
- Allows for tracking of Object creation



Milestone 5 – Cache Manager (w/sort and range)



Recursion

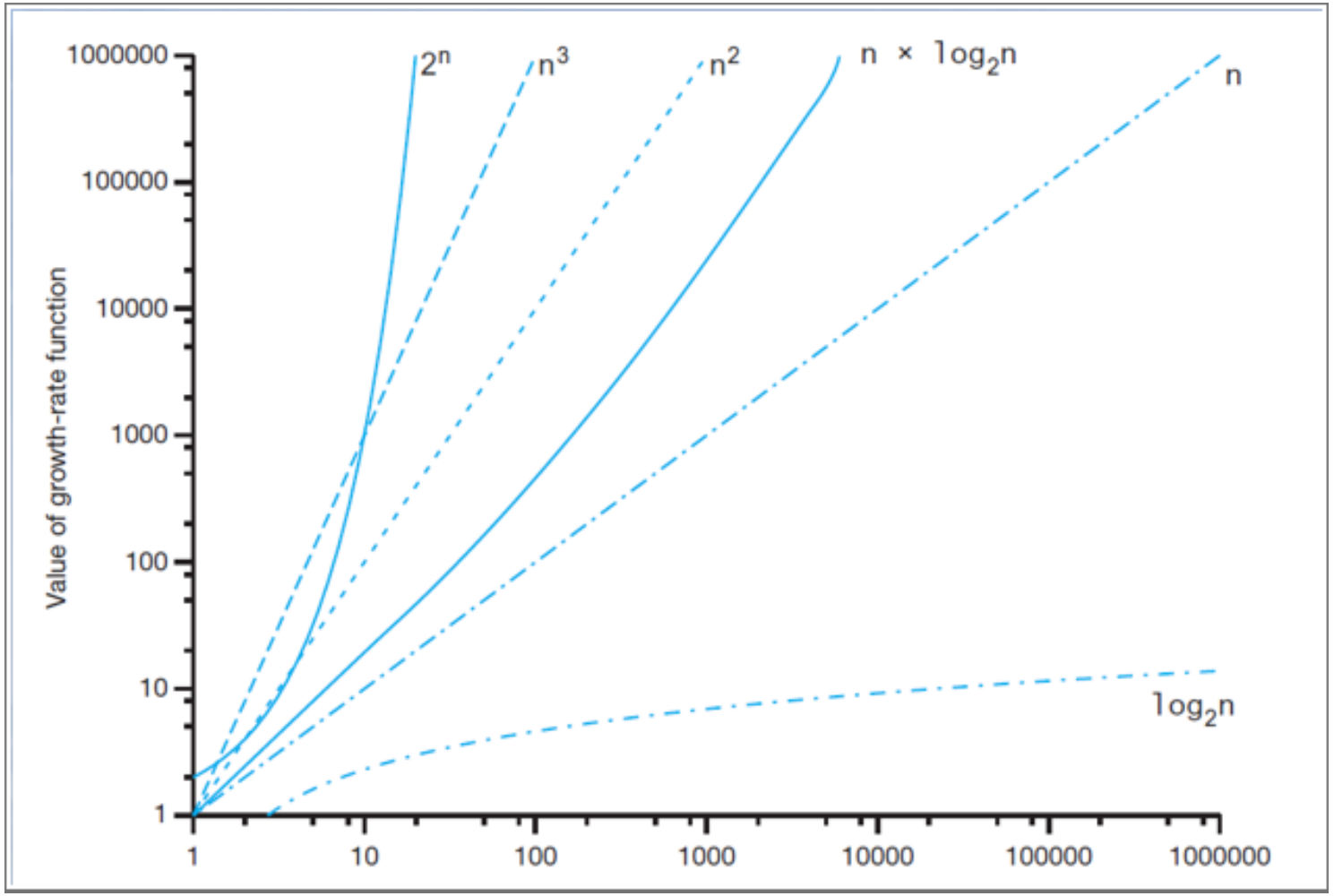
- Technique where a function/method calls itself to break a problem into smaller identical problems
- Base case (to exit)
- Eventually, one of the smaller problems must be the base case
- Generally, iterative solutions are simpler/more performant
- Primary tree traversal technique (unknown number of levels)

Algorithm Efficiency

- Methodology to measure and describe efficiency
- Memorize cost of different operations of different data structures
- $O(1) < O(\log n) < O(n) < O(n\log n) < O(n^2) < O(n^3) < O(2^n)$

Analysis and Big O Notation

- Learn how to read this graph:



Additional C++ Topics

- Exception handling
  - When to throw exceptions
  - Main Exception types

- Overloading
  - Operator overloading
  - How/when to use them

Additional C++ Topics (cont)

- Smart Pointers
  - Smart Pointers vs GC

- Standard Template Library (STL)
  - STL vs generics
  - Containers
  - Iterators

Testing

- Unit Testing
- Integration Testing
- Black Box Testing
- White Box Testing